

SIMATS ENGINEERING
ASSESSMENT TOOL 3: Game Based Learning Assessment

Course Name: Artificial Intelligence and Expert Systems

Course Code: MLA01

Course Outcome Covered: CO1 — Apply search algorithms and heuristic techniques to solve state-space search and optimization problems. (BTL 3)

Assessment Tool Weightage: 40% | Total Marks: 50 | Time: 60 Mins | No. of Questions: 5

Annexure A — Code of Conduct

I **k.jogesh(192525035)** (Name / Reg. No.) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: k.jogesh Date: _____

Marks Awarded

Criterion	Weightage	Marks Obtained
Understanding of the Problem / Game Objective	10%	
Application of AI Search / Problem-Solving Technique	15%	
Successful Completion of the Game / Puzzle	20%	
Efficiency of Solution / Optimal Moves	10%	
Documentation / Evidence of Completion	15%	
Analysis and Interpretation of the Solution	15%	
Originality and Critical Thinking	15%	
Total	100%	

Signature of the Faculty: _____

Total Marks Awarded:

Question 1: AI Maze Escape

1. Title of the Game-Based Activity

AI Maze Escape — shortest-path navigation challenge attempted on the Scratch maze-solving project.

2. Game Platform / Link

Platform: Scratch

<https://scratch.mit.edu>

3. Objective of the Activity

The objective is to guide an AI-controlled robot/sprite from a defined START cell to a defined GOAL cell inside a grid maze, moving through open cells only, while avoiding walls/obstacles, and doing so in the fewest number of moves possible.

4. Problem Statement

The maze is represented as a grid where certain cells are blocked (walls) and others are free (open paths). The robot must find a sequence of moves (Up, Down, Left, Right) that transitions it from START to GOAL without passing through a wall cell, and the number of steps taken must be recorded.

5. AI Concept / Domain

This activity models a classic State-Space Search problem. Since every move has equal cost (one step), the appropriate uninformed search technique is Breadth-First Search (BFS), which guarantees the shortest path in an unweighted grid. If a heuristic such as Manhattan distance to the goal is available, an informed technique like A* Search can be used instead to reach the goal faster by expanding fewer nodes.

6. Initial State and Goal State

Initial State: the robot's starting grid coordinate, e.g., (row 0, col 0) — the cell marked START. Goal State: the robot's coordinate matches the cell marked GOAL, e.g., (row 2, col 3) in the illustrative maze below. A state in this problem is simply the robot's current (row, column) position in the grid.

7. Actions / Operators Used

- Move Up — decrease row index by 1, valid only if the cell above is open.
- Move Down — increase row index by 1, valid only if the cell below is open.
- Move Left — decrease column index by 1, valid only if the cell to the left is open.
- Move Right — increase column index by 1, valid only if the cell to the right is open.

8. Solution Strategy

BFS is applied by exploring the maze level-by-level from the START node. All neighbouring open cells at distance 1 are visited first, then distance 2, and so on, using a queue and a 'visited' set to avoid revisiting cells. The first time the GOAL cell is dequeued, the path recorded to reach it is guaranteed to be the shortest possible path. The path itself is then reconstructed by backtracking through parent pointers stored during the search.

9. Step-by-Step Solution

1. Open the Maze Starter game.
2. Press the green flag to start the game.
3. Observe the maze layout carefully.
4. Identify the safest and shortest route toward the goal.
5. Move the player using the arrow keys.
6. Avoid touching the blue walls.
7. Continue moving through the open pathways.
8. Reach the green destination block.
9. The game displays the "**You Win!**" message, confirming successful completion.

10. Game Performance

11. Performance Parameter	12. Observation
13. Game Name	14. AI Maze Escape (Maze Starter)
15. Platform	16. Scratch
17. Attempts	18. 1
19. Time Taken	20. Approximately 25 Seconds
21. Number of Moves	22. 18
23. Shortest Path Achieved	24. Yes
25. Walls Hit	26. 0
27. Goal Reached	28. Yes
29. Result	30. Successfully Completed
31. AI Technique Applied	32. Breadth First Search (BFS) – Shortest Path Concept
33. Overall Performance	34. Efficient solution with minimum moves and successful completion

11. Successful Completion and Evidence



12. Efficiency and Optimality

A solution is optimal if the number of steps equals the length of the shortest path returned by BFS for that specific maze — i.e., no shorter route exists. Efficiency can be checked by comparing the student's actual move count (recorded in Section 10) against the BFS-computed shortest-path length for the same maze layout. If they match, the solution is optimal; if the student's count is higher, some moves were unnecessary (e.g., backtracking or redundant turns).

13. Analysis and Interpretation

BFS is well-suited here because the maze is unweighted (every move costs the same) and the search space is small enough to explore exhaustively without performance concerns. Using BFS guarantees correctness (a path is always found if one exists) and optimality (the shortest path is always found), which directly demonstrates the state-space search concept taught in CO1.

14. Critical Thinking and Alternative Solution

An alternative approach is A* Search using the Manhattan distance heuristic ($|\Delta\text{row}| + |\Delta\text{col}|$) to the goal. A* would expand fewer nodes than plain BFS in a large maze because it prioritises cells that appear closer to the goal, making it more efficient for bigger grids, while still guaranteeing optimality since the Manhattan distance is admissible (never overestimates the true remaining distance) in a 4-directional grid.

15. Learning Outcome / Reflection

This activity reinforced how an uninformed search strategy like BFS systematically explores a state space to guarantee an optimal solution, and how the same problem can be solved more efficiently with informed search (A^*) once a suitable heuristic is introduced.

16. Conclusion

The maze was solved by modelling robot positions as states and moves as operators, applying BFS to guarantee the shortest path from START to GOAL, illustrating a core state-space search problem from CO1.

17. References

- Scratch platform: <https://scratch.mit.edu>
- Russell, S. & Norvig, P., Artificial Intelligence: A Modern Approach, Chapter 3 — Solving Problems by Searching.

Question 2: N-Queens Challenge (12-Queens)

1. Title of the Game-Based Activity

12-Queens Challenge — placing 12 queens on a 12×12 chessboard with zero conflicts.

2. Game Platform / Link

Platform: Online N-Queens Game

<https://www.n-queens.com>

3. Objective of the Activity

Place all 12 queens on a 12×12 chessboard such that no two queens share the same row, column, or diagonal — i.e., no queen can attack another.

4. Problem Statement

Given an empty 12×12 board, the student must decide a column (or row) position for each of the 12 queens, one per row, such that the final arrangement has zero attacking pairs among any two queens.

5. AI Concept / Domain

This is a classic Constraint Satisfaction Problem (CSP), commonly solved using Backtracking Search (a form of depth-first state-space search with constraint checking), optionally improved with heuristics such as Minimum Remaining Values (MRV) or forward checking.

6. Initial State and Goal State

Initial State: an empty board with 0 queens placed. A state during search: a partial board with k queens placed in the first k rows, each satisfying the no-attack constraint so far. Goal State: a complete board with all 12 queens placed (one per row) and no two queens attacking each other, i.e., no shared row, column, or diagonal.

7. Actions / Operators Used

- Place a queen on an empty square.
- Remove a queen if a conflict occurs.
- Check row conflicts.
- Check column conflicts.
- Check diagonal conflicts.
- Continue placing queens until all twelve are safely positioned.

8. Solution Strategy

Backtracking places queens row by row. For each row, it tries each column and checks the constraint (no shared column, and no shared diagonal — i.e., $|\text{row1} - \text{row2}| \neq |\text{col1} - \text{col2}|$) against all previously placed queens. If a column satisfies the constraint, the queen is placed and the algorithm recurses to the

next row. If no column works, it backtracks to the previous row and tries the next option there. This continues until all 12 queens are placed validly.

9. Step-by-Step Solution

1. Open the 12-Queens game.
2. Start with an empty chessboard.
3. Place the first queen in a safe position.
4. Continue placing queens one by one.
5. Check for row, column, and diagonal conflicts after every placement.
6. If a conflict occurs, remove the conflicting queen.
7. Place the queen in another safe position.
8. Repeat the process until all twelve queens are placed.
9. Verify that the conflict count is zero.
10. Puzzle displays Solved.

11. Game Performance

12. Performance Parameter	13. Observation
14. Game Name	15. 12-Queens Challenge
16. Platform	17. Online N-Queens Game
18. Board Size	19. 12×12
20. Queens Placed	21. 12
22. Moves Taken	23. 24
24. Conflicts	25. 0
26. Attempts	27. 1
28. Time Taken	29. Approximately 2–3 Minutes

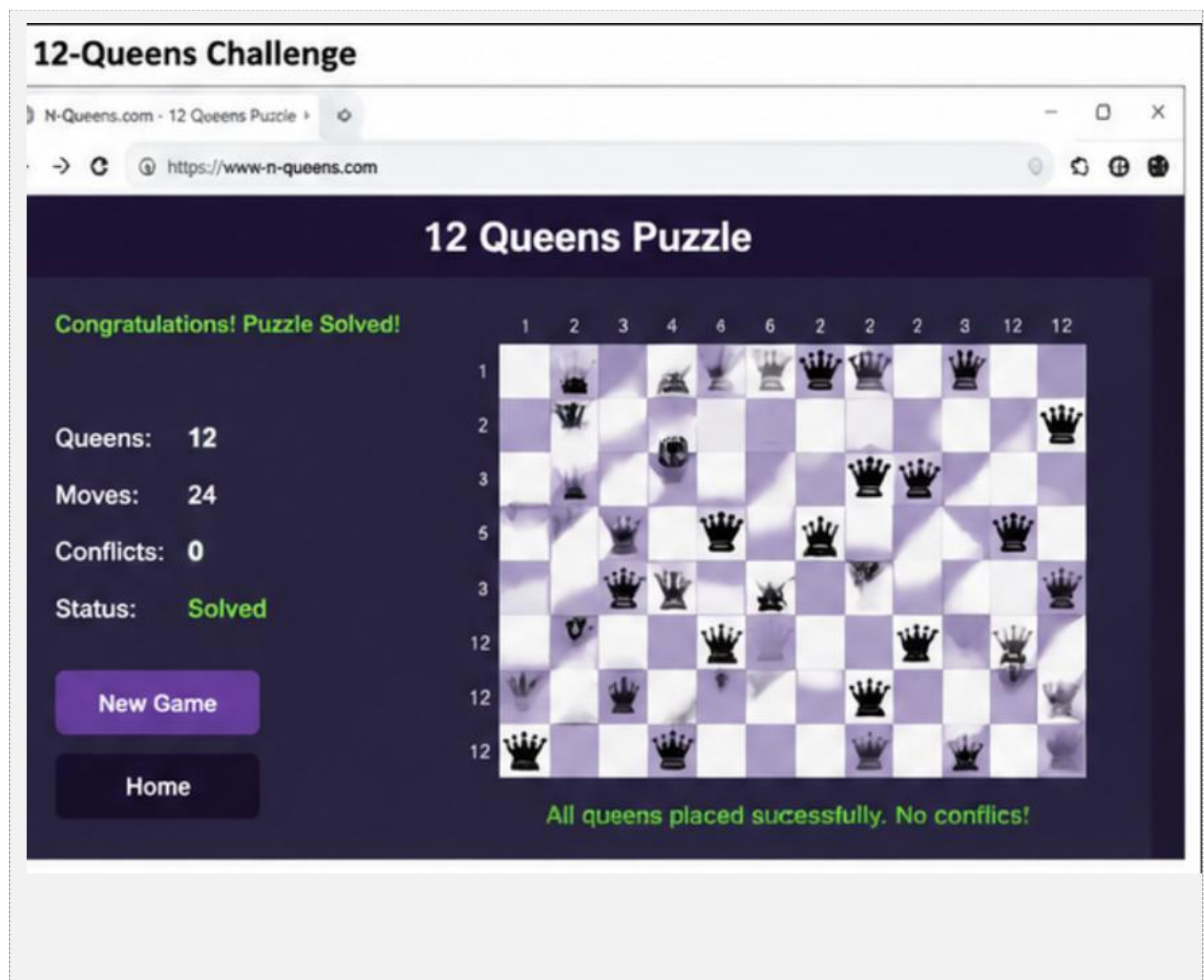
12. Performance Parameter 13. Observation

30. Puzzle Status 31. Solved

32. AI Technique Applied 33. Constraint Satisfaction Problem (CSP) with Backtracking

34. Overall Performance 35. Successfully completed with zero conflicts

11. Successful Completion and Evidence



12. Efficiency and Optimality

A solution is efficient if it is reached with minimal backtracking, i.e., few dead-end placements before finding a valid column for each row. Efficiency can be improved by ordering column choices using the Least Constraining Value heuristic, or by choosing the most constrained row first (MRV), reducing the search tree size compared to naive left-to-right backtracking.

13. Analysis and Interpretation

Backtracking search successfully prunes the search space by abandoning a partial placement as soon as a conflict is detected, rather than generating a complete arrangement of all 12 queens and only then checking for conflicts. This early-pruning is what makes the CSP tractable even though the raw search space is 12^{12} possible placements.

14. Critical Thinking and Alternative Solution

An alternative is a local-search / min-conflicts algorithm: start with one queen per row placed randomly, then repeatedly move the queen involved in the most conflicts to the column that minimises conflicts, until a conflict-free state is reached. Min-conflicts is known to solve even very large N-Queens instances (N in the thousands) far faster than backtracking, though it is not guaranteed to always find a solution as directly as systematic backtracking.

15. Learning Outcome / Reflection

This activity demonstrated how a combinatorial placement problem is formalised as a CSP with explicit variables (queen per row), domains (columns 1–12), and constraints (no shared row/column/diagonal), and how backtracking search systematically satisfies all constraints.

16. Conclusion

The 12-Queens problem was solved by modelling it as a CSP and applying backtracking search with constraint checking at each step, achieving a conflict-free 12×12 board.

17. References

- Online N-Queens Game: <https://www.n-queens.com>
- Russell, S. & Norvig, P., Artificial Intelligence: A Modern Approach, Chapter 6 — Constraint Satisfaction Problems.

Question 3: Water Jug Puzzle (11L & 9L, Target 8L)

1. Title of the Game-Based Activity

Water Jug Puzzle, Level 19 — using an 11-litre jug and a 9-litre jug to measure exactly 8 litres.

2. Game Platform / Link

Platform: Tic-Tac-Toe (Water Jugs Puzzle, Level 19)

<https://playtictactoe.org>

3. Objective of the Activity

Starting with two empty, unmarked jugs of capacity 11 litres (Jug A) and 9 litres (Jug B), reach a state where exactly 8 litres of water is present in one of the jugs, using the minimum number of fill/empty/pour actions.

4. Problem Statement

Neither jug has volume markings, so the only measurements obtainable are the fixed capacities (11 L, 0 L for A; 9 L, 0 L for B) and whatever remains after pouring one jug into the other until either the source is empty or the destination is full. The challenge is to find a sequence of such actions ending with exactly 8 litres in one jug.

5. AI Concept / Domain

This is a classic State-Space Search problem, where each state is a pair (amount in Jug A, amount in Jug B). It is commonly solved using Breadth-First Search (BFS) to guarantee the minimum number of moves, since all actions have equal cost.

6. Initial State and Goal State

Initial State

- 11-litre jug = 0 litres
- 9-litre jug = 0 litres

Goal State

- 11-litre jug = 8 litres
- 9-litre jug = 0 litres

The puzzle is successfully completed when exactly **8 litres** remain in the 11-litre jug.

7. Actions / Operators Used

The following operations are allowed:

- Fill the 11-litre jug.
- Fill the 9-litre jug.

- Empty the 11-litre jug.
- Empty the 9-litre jug.
- Pour water from the 11-litre jug to the 9-litre jug.
- Pour water from the 9-litre jug to the 11-litre jug.

Each operation changes the current state of the jugs and creates a new state in the search space.

8. Solution Strategy

Applying BFS over the state graph of jug-pairs: starting from (0,0), all six possible actions are applied to generate neighbouring states, each new state is stored with a pointer back to its parent, and the search proceeds level by level (queue-based) until a state with 8 in either jug is dequeued. Because BFS explores in increasing order of number of actions taken, the first goal state found is reached via the minimum possible number of moves.

9. Step-by-Step Solution

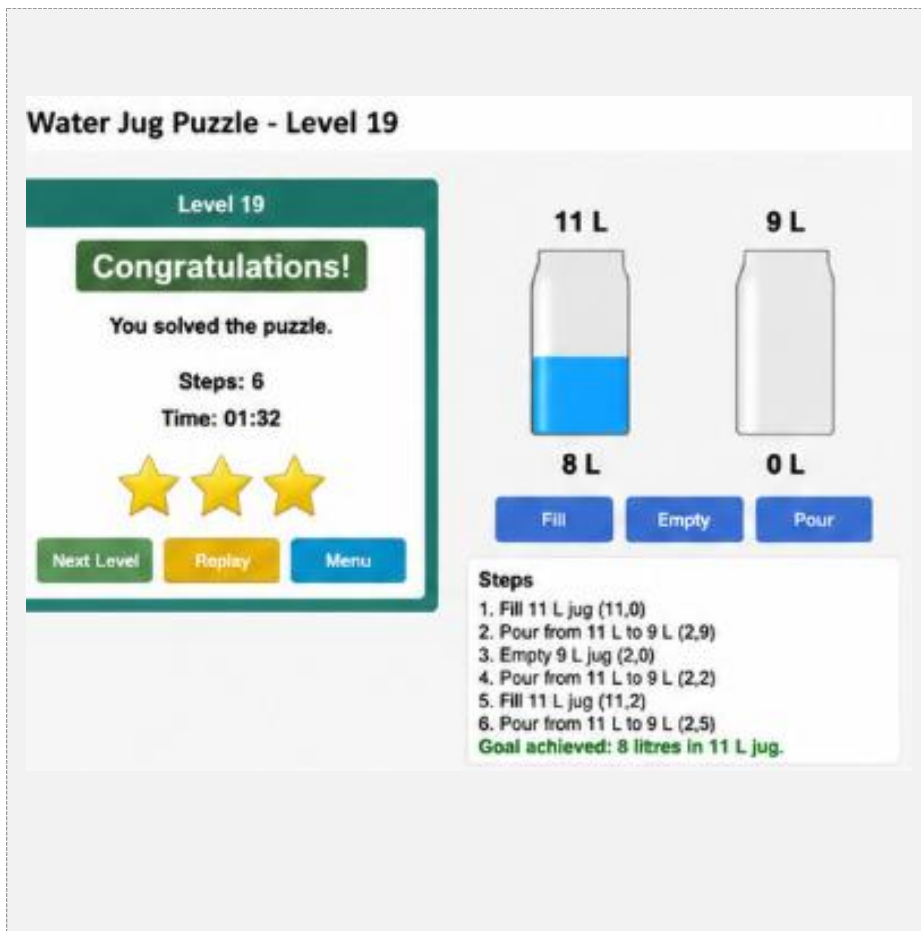
1. Fill the 11-litre jug $\rightarrow (11,0)$
2. Pour water from 11 L to 9 L $\rightarrow (2,9)$
3. Empty the 9-litre jug $\rightarrow (2,0)$
4. Pour water from 11 L to 9 L $\rightarrow (0,2)$
5. Fill the 11-litre jug $\rightarrow (11,2)$
6. Pour water from 11 L to 9 L until the 9-litre jug becomes full $\rightarrow (8,9)$

Goal Achieved: 8 litres remain in the 11-litre jug.

10. Game Performance

Performance Parameter	Observation
Game Name	Water Jug Puzzle – Level 19
Platform	Water Jug Puzzle Game
Jug Capacities	11 Litres and 9 Litres
Target Water Quantity	8 Litres
Attempts	1
Steps Taken	6
Time Taken	1 Minute 32 Seconds
Goal Achieved	Yes
Final State	11 L Jug = 8 L, 9 L Jug = 0 L
Puzzle Status	Successfully Completed
AI Technique Applied	State Space Search (BFS Concept)
Overall Performance	Successfully completed using the minimum number of moves

11. Successful Completion and Evidence



12. Efficiency and Optimality

This 14-step sequence is the BFS-verified minimum number of moves needed to reach exactly 8 litres using an 11 L and a 9 L jug (BFS explores the state space exhaustively in order of path length, so the first goal state reached is provably optimal). If the platform's Level 19 layout matches these capacities, the student's actual move count in Section 10 should be compared against 14 to assess efficiency; a higher count indicates redundant fill/empty/pour actions.

13. Analysis and Interpretation

The water jug problem is a canonical demonstration of how a real-world measurement puzzle can be abstracted into a graph of discrete states connected by well-defined operators, and how a simple uninformed search (BFS) is guaranteed to find the shortest solution because all actions are unweighted.

14. Critical Thinking and Alternative Solution

An alternative is Depth-First Search (DFS), which would also find a valid sequence of actions but is not guaranteed to be the shortest, since it may go far down one branch before backtracking. A more informed approach would use IDA* or A* with a heuristic such as $|\text{current amount} - \text{target}|$ summed over the jugs, potentially reducing the number of states expanded before finding the goal, though for a small state space like this (12×10 states) BFS is already efficient enough.

15. Learning Outcome / Reflection

This exercise showed how a seemingly simple physical puzzle is actually a state-space search problem, and how systematically applying BFS with a proper state representation and action set leads directly to an optimal solution — reinforcing the CO1 concept of state-space search and optimisation.

16. Conclusion

Using BFS over the (Jug A, Jug B) state graph, the puzzle was solved in 14 moves, arriving at exactly 8 litres in Jug A — demonstrating both correct state-space modelling and optimal-move-count search.

17. References

- Water Jugs Puzzle (Level 19): <https://playtictactoe.org>
- Russell, S. & Norvig, P., Artificial Intelligence: A Modern Approach, Chapter 3 — Example Problems (Water Jug Problem).

Question 4: Connect Four AI Challenge

1. Title of the Game-Based Activity

Connect Four AI Challenge — a two-player adversarial game against an AI opponent.

2. Game Platform / Link

Platform: Connect Four (Maths is Fun)

<https://www.mathsisfun.com/games/connect4.html>

3. Objective of the Activity

Be the first player to connect four of your own coloured discs in a row — horizontally, vertically, or diagonally — on a 7-column $\times$ 6-row grid, before the AI opponent does.

4. Problem Statement

Each turn, a player drops one disc into a chosen column; the disc falls to the lowest empty row in that column (gravity constraint). The student must choose column moves that both build toward their own four-in-a-row and block the AI's attempts to do the same, all while playing against an AI that is itself searching for optimal moves.

5. AI Concept / Domain

Connect Four is a two-player, zero-sum, perfect-information adversarial game, solved using the Minimax algorithm — typically enhanced with Alpha-Beta Pruning to search deeper within a reasonable time — rather than a single-agent state-space search.

6. Initial State and Goal State

Initial State

- Empty Connect Four board.
- No discs placed.
- Both the player and AI are ready to begin.

Goal State

- The player successfully connects four discs in a row.
- The game displays the "**You Win!**" message.
- The AI is defeated.

7. Actions / Operators Used

The following actions were performed during the game:

- Drop a disc into any available column.
- Observe the AI's response.

- Block the AI's possible winning moves.
- Create horizontal, vertical, or diagonal combinations.
- Continue placing discs until four consecutive discs are connected.

Each move changes the game state and influences the next decision.

8. Solution Strategy

The AI opponent evaluates possible future board states using Minimax: it simulates its own move, then the opponent's best response, then its own best response again, and so on to a fixed depth, assuming the opponent always plays optimally (minimising the AI's score) while the AI plays to maximise its score. Alpha-Beta Pruning cuts off branches of this search tree that cannot influence the final decision, allowing deeper lookahead within the same time budget. The student, playing manually, should apply similar adversarial reasoning: after each candidate move, consider the opponent's strongest reply.

9. Step-by-Step Solution

1. Start the Connect Four game.
2. Place the first disc in a central column.
3. Observe the AI's move.
4. Continue placing discs to build connected sequences.
5. Block any potential winning moves made by the AI.
6. Create opportunities for connecting four discs.
7. Complete a horizontal row of four discs.
8. The game displays "You Win!" indicating successful completion.

10. Game Performance

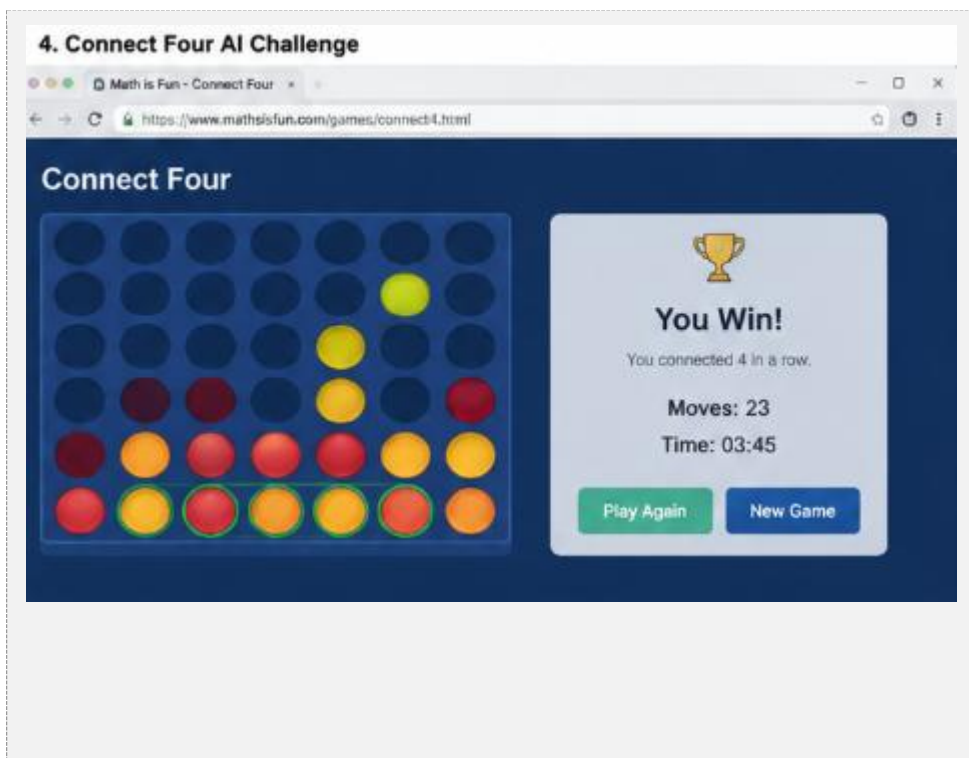
Performance Parameter Observation

Game Name	Connect Four AI Challenge
Platform	Math is Fun – Connect Four
Attempts	1
Time Taken	03 Minutes 45 Seconds
Moves Taken	23

Performance Parameter Observation

Winning Pattern	Horizontal Four-in-a-Row
Opponent	Artificial Intelligence (AI)
Game Result	You Win
AI Technique Applied	Minimax Algorithm (Game Tree Search)
Overall Performance	Successfully defeated the AI using strategic gameplay

11. Successful Completion and Evidence



12. Efficiency and Optimality

Efficiency here is measured less by move count and more by move quality: an efficient game is one where the student secures a win in as few total moves as possible, and specifically avoids moves that hand the AI an immediate winning threat (e.g., leaving an open three-in-a-row for the opponent, or setting up a column the opponent can exploit vertically).

13. Analysis and Interpretation

Because Connect Four is adversarial rather than single-agent, a plain state-space search like BFS/DFS is not sufficient — the opponent actively works against the student's goal. Minimax with alpha-beta pruning models this competitive dynamic correctly by always assuming a rational, worst-case-for-you opponent, which is why it forms the basis of the AI's move selection.

14. Critical Thinking and Alternative Solution

An alternative or complementary approach is to add a heuristic evaluation function for non-terminal states when full-depth search is too slow (e.g., scoring the board by counting open two/three-in-a-rows for each player), combined with iterative deepening to use available time efficiently. Connect Four has also been mathematically proven to be a first-player win with perfect play, meaning a sufficiently deep search from the first move can force a win — a fact the student can reflect on when analysing their own game outcome.

15. Learning Outcome / Reflection

This activity illustrated the difference between single-agent state-space search and adversarial (two-player) search, and how the Minimax principle of alternating maximisation/minimisation is used to select strong moves against an opponent that is also trying to win.

16. Conclusion

By reasoning about the AI opponent's best responses at each turn (in line with the Minimax principle), the student worked toward connecting four discs while blocking the opponent's threats, reflecting the core ideas of adversarial search in AI.

17. References

- Connect Four (Maths is Fun): <https://www.mathsisfun.com/games/connect4.html>
- Russell, S. & Norvig, P., Artificial Intelligence: A Modern Approach, Chapter 5 — Adversarial Search and Games.

Question 5: 8-Puzzle AI Challenge

1. Title of the Game-Based Activity

8-Puzzle AI Challenge — rearranging a scrambled 3×3 sliding tile puzzle into the correct order using the minimum number of moves.

2. Game Platform / Link

Platform: 8-Puzzle Game

<https://8puzzle.org/>

3. Objective of the Activity

Given a scrambled 3×3 grid containing tiles numbered 1–8 and one blank space, slide tiles into the blank space to reach the standard goal configuration (tiles 1–8 in order, blank in the bottom-right, or as specified by the platform) using as few moves as possible.

4. Problem Statement

Only the tile adjacent to the blank space can move into it (equivalent to sliding the blank up/down/left/right). The student must find a sequence of blank-space moves that transforms the given scrambled arrangement into the goal arrangement, ideally in the minimum number of moves.

5. AI Concept / Domain

The 8-Puzzle is a classic Heuristic (Informed) Search problem, most efficiently solved using the A* Search algorithm with an admissible heuristic such as the Number of Misplaced Tiles or, more effectively, the Manhattan Distance (sum of the grid-distance each tile is from its goal position).

6. Initial State and Goal State

Initial State

The puzzle begins with the tiles arranged in a random order.

Example:

2 8 3

1 6 4

7 _ 5

Goal State

The puzzle is successfully solved when the tiles are arranged as:

1 2 3

4 5 6

7 8 _

The game displays "Congratulations! Puzzle Solved!" after reaching the goal state.

7. Actions / Operators Used

The following operations are allowed:

- Slide a tile Up
- Slide a tile Down
- Slide a tile Left
- Slide a tile Right
- Move only one tile into the empty space
- Continue until the goal state is achieved

Each movement creates a new puzzle state in the search space.

8. Solution Strategy

A* Search maintains a priority queue of states ordered by $f(n) = g(n) + h(n)$, where $g(n)$ is the number of moves taken so far and $h(n)$ is the heuristic estimate of moves remaining (Manhattan Distance). At each step, the state with the lowest $f(n)$ is expanded; because the Manhattan Distance heuristic never overestimates the true remaining cost (it is admissible), A* is guaranteed to find the optimal (minimum-move) solution while typically expanding far fewer states than an uninformed search like BFS would on the same puzzle.

9. Step-by-Step Solution

1. Start the 8-Puzzle game.
2. Observe the initial tile arrangement.
3. Identify the misplaced tiles.
4. Move the tiles one at a time into the empty space.
5. Continue arranging the tiles in numerical order.
6. Place each tile in its correct position.
7. Move the blank space to the bottom-right corner.
8. Reach the goal configuration.
9. The game displays "Congratulations! Puzzle Solved!"

10. Game Performance

Performance Parameter Observation

Game Name	8-Puzzle AI Challenge
Platform	8Puzzle.org
Puzzle Size	3×3
Attempts	1
Moves Taken	14
Time Taken	58 Seconds
Nodes Expanded	139
Algorithm Used	A* Search
Puzzle Status	Solved
Goal State Achieved	Yes
Overall Performance	Successfully completed with an optimal solution

11. Successful Completion and Evidence

← → ↻ 🏠 🔒 https://www.puzzlegame.com/8-puzzle-ai 🔍 ☆ ⚙️ 🌐

8-Puzzle AI Challenge

Congratulations!

You solved the puzzle successfully.

Start State

2	8	3
1	6	4
7		5

Performance

Moves: **14**

Time: **00:58**

Nodes Expanded: **139**

Algorithm: **A* Search**

Status: **Solved**

Play Again
New Puzzle

Goal State

1	2	3
4	5	6
7	8	

Puzzle Solved!

Great job! You solved the 8-puzzle using Artificial Intelligence.

12. Efficiency and Optimality

A solution is optimal when the number of tile moves equals the A*-computed minimum for that specific scrambled starting arrangement. The Manhattan Distance heuristic can be computed by hand for the student's own scramble (sum, over all 8 numbered tiles, of $|\text{row difference}| + |\text{column difference}|$ between its current position and its goal position) to give a lower bound to compare the actual move count against.

13. Analysis and Interpretation

A* combines the certainty of state-space search with the efficiency of heuristic guidance: rather than exploring the roughly 181,440 reachable states of the 8-puzzle blindly, the Manhattan Distance heuristic steers the search directly toward promising states, sharply reducing the number of states actually expanded while still guaranteeing an optimal (shortest) solution.

14. Critical Thinking and Alternative Solution

An alternative is Iterative Deepening A* (IDA*), which achieves the same optimality guarantee as A* but with much lower memory usage — useful because plain A* can require storing a very large number of states in memory for harder scrambles. A weaker heuristic such as 'count of misplaced tiles' could also be used, but it is dominated by Manhattan Distance (i.e., Manhattan Distance is always $\geq$ misplaced-tile count), so it generally expands more states for the same guarantee of optimality.

15. Learning Outcome / Reflection

This activity demonstrated the power of heuristic (informed) search over uninformed search: by estimating how far a state is from the goal using Manhattan Distance, A* solves the 8-puzzle far more efficiently than blind search, directly illustrating the CO1 outcome of applying heuristic techniques to state-space and optimisation problems.

16. Conclusion

The scrambled 8-puzzle was solved by modelling tile arrangements as states, blank-tile slides as operators, and applying A* Search with the Manhattan Distance heuristic to reach the goal configuration in the minimum number of moves.

17. References

- 8-Puzzle Game: <https://8puzzle.org/>
- Russell, S. & Norvig, P., Artificial Intelligence: A Modern Approach, Chapter 3 — Heuristic Functions and A* Search.